NVIDIA Dynamo

HPE Career Preview Program

Agenda

- Introduction to NVIDIA Dynamo
- Aggregated Serving
- Disaggregated Serving
- AI Perf Benchmarking and Performance Analysis
- KV Buffer Manager (KVBM)
- LMCache Integration and Prompt Reuse
- AI Configurator
- Conclusion

Introduction to NVIDIA Dynamo

What is NVIDIA Dynamo?

- NVIDIA Dynamo is a distributed inference framework designed for serving Large Language Models (LLMs) efficiently.
- It enables scalable and high-performance AI inference across one or more GPUs.
- Dynamo separates different inference components, making deployment modular and easier to scale.
- It integrates with vLLM to provide fast token generation, efficient GPU utilization, and optimized memory management.

Key Features

- High-performance LLM serving
- Aggregated and Disaggregated Serving
- KV Cache Management
- AI Perf Benchmarking
- Scalable Multi-GPU Inference

Aggregated Serving using NVIDIA Dynamo

Aggregated Serving Overview

What is Aggregated Serving?

- Aggregated Serving is the simplest deployment architecture in NVIDIA Dynamo.
- A single vLLM worker executes the complete inference pipeline.
- The same worker performs:
  - Prompt Processing (Prefill)
  - Token Generation (Decode)
- The model remains loaded on a single NVIDIA GPU throughout inference.

System Components

- Client Application
- Dynamo Frontend API Server
- Single Dynamo vLLM Worker
- Meta-Llama-3.1-8B-Instruct Model
- NVIDIA A100 GPU

Request Flow

$$\text{Client} \rightarrow \text{Frontend} \rightarrow \text{Worker} \rightarrow \text{GPU} \rightarrow \text{Response}$$

Launching Dynamo Services

1. Start the Frontend

```
python -m dynamo.frontend \
  --discovery-backend file \
  --http-port 8150
```

The frontend acts as the API gateway and routes incoming requests to registered workers.

2. Start the vLLM Worker

```
HF_TOKEN="YOUR_HF_TOKEN"
CUDA_VISIBLE_DEVICES=5

python -m dynamo.vllm \
  --model meta-llama/Meta-Llama-3.1-8B-Instruct \
  --discovery-backend file \
  --gpu-memory-utilization 0.9 \
  --kv-events-config '{"enable_kv_cache_events":false}'
```

Testing Inference

Sending an Inference Request

```
curl -s http://localhost:8150/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{ ... }'
```

Inference Workflow

- Client sends a request using the REST API.
- Dynamo Frontend receives the request.
- Frontend forwards it to the registered vLLM worker.
- The worker performs Prompt Processing (Prefill) and Token Generation (Decode).
- The generated response is returned in JSON format.

Result
The successful JSON response confirms that the model was loaded correctly and the Aggregate Serving deployment is working as expected.

Observations and Key Takeaways

Observations

- A single worker performs both prompt processing and decoding.
- No KV-cache transfer or synchronization overhead exists.
- Simple architecture enables easy deployment and baseline evaluation.
- GPU memory utilization directly impacts model capacity and performance.
- AI Perf provides detailed latency and throughput statistics.

Key Learning

Aggregated Serving provides an efficient baseline deployment for LLM inference. It establishes the foundation for understanding more advanced architectures such as Disaggregated Serving.

Takeaway

Understanding Aggregated Serving established the foundation for learning Prefill/Decode separation, KV-cache management, and scalable distributed AI inference.

Disaggregated Serving using NVIDIA Dynamo

From Aggregated to Disaggregated Serving

Limitation of Aggregated Serving

- A single worker performs Prefill (compute-bound) and Decode (memory-bandwidth-bound) sequentially.
- These two phases have fundamentally different resource profiles, so co-locating them causes GPU contention.
- Decode requests are forced to wait behind long prefill operations, increasing tail latency.
- GPU utilization cannot be independently tuned for either phase.

What Disaggregated Serving Changes

- Prefill and Decode are split onto physically separate workers, each with dedicated GPU resources.
- The KV cache generated during Prefill must now be transferred across nodes to the Decode worker.
- A dedicated transport layer (NIXL) and event-driven cache signaling replace in-process memory sharing.

Key Shift

Aggregated Serving optimizes for simplicity; Disaggregated Serving optimizes for throughput and latency by decoupling phase-specific compute from cache-transfer logistics.

Disaggregated Serving Overview

What is Disaggregated Serving?

- Splits the inference pipeline into two independently deployed Dynamo vLLM workers.
- Prefill Worker: handles Prompt Processing only (`--disaggregation-mode prefill`).
- Decode Worker: handles Token Generation only (`--disaggregation-mode decode`).
- Each worker runs on a distinct GPU, isolated via `CUDA_VISIBLE_DEVICES`.

System Components

- Client Application
- Dynamo Frontend API Server
- Dynamo vLLM Prefill Workers
- Dynamo vLLM Decode Workers
- NIXL KV Transfer Connector
- meta-llama/Llama-3.1-8B-Instruct

Request Flow

Client → Frontend → Prefill Worker → KV Cache Transfer (NIXL) → Decode Worker → Response

Orchestration: How the Pieces Interconnect

Discovery and Routing

- Both workers register with the same file-based discovery backend (`--discovery-backend file`), allowing the Frontend to distinguish Prefill-capable and Decode-capable workers.
- The Frontend routes an incoming request first to a Prefill worker, then hands off the resulting KV cache reference to a Decode worker.

KV Cache Transfer via NIXL

- `NixlConnector` implements point-to-point GPU memory transfer of KV cache blocks, bypassing the CPU/network stack overhead of a naive copy.
- The `VLLM_NIXL_SIDE_CHANNEL_PORT` on the Prefill worker exposes the endpoint that the Decode worker connects to when pulling cache blocks.
- `enable_kv_cache_events` allows fine-grained, block-level notification, so the Decode worker begins generation as soon as required blocks land rather than waiting on a full-transfer barrier.

Result

Prefill computation and Decode computation now execute concurrently on separate GPUs, with KV cache state bridged asynchronously via NIXL rather than sharing memory in-process.

Observations and Key Takeaways

Observations

- Splitting Prefill and Decode onto separate GPUs removes head-of-line blocking between the two phases.
- KV cache transfer introduces network/interconnect latency that did not exist in Aggregated Serving; NIXL and event-driven signaling are designed specifically to minimize this overhead.
- Independent scaling becomes possible: additional Prefill or Decode workers can be added based on which phase is the current bottleneck.

Key Learning

Disaggregated Serving trades the simplicity of Aggregated Serving for the ability to independently provision, scale, and optimize the compute-bound Prefill phase and the bandwidth-bound Decode phase, at the cost of managing cross-node KV cache transfer.

Takeaway

Understanding Disaggregated Serving establishes the foundation for evaluating the performance trade-offs between phase separation, GPU utilization, and KV-cache transfer overhead.

Performance Comparison Across Disaggregation Ratios

Benchmark Setup

- Measured with `aiperf` on Meta-Llama-3.1-8B-Instruct, concurrency 100, 500 requests per run, across all four disaggregation ratios: 1P-1D, 1P-2D, 2P-1D, and 2P-2D.
- Two fixed synthetic workloads applied to every configuration, with output length held constant at ≈256 tokens: **highISL** (input ≈16000 tokens – prefill-dominated) and **lowISL** (input ≈500 tokens – lighter, more balanced prefill/decode load).

highISL workload (ISL ≈16000, OSL ≈256)

Config	Req. Thpt. (req/s)	Out. Tok. Thpt. (tok/s)	Avg. Latency (s)	TTFT avg (s)	ITL avg (ms)
1P-1D	0.10	25.6	927.8	917.7	39.3
1P-2D	0.09	23.0	1032.1	1026.0	23.9
2P-1D	0.21	53.5	449.3	432.2	67.1
2P-2D	0.23	59.6	401.7	396.5	20.6

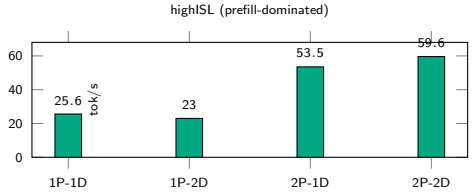
lowISL workload (ISL ≈500, OSL ≈256)

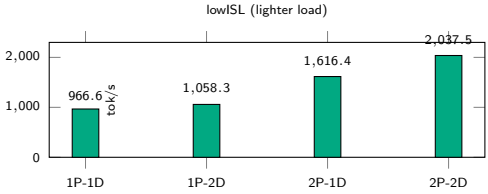
Config	Req. Thpt. (req/s)	Out. Tok. Thpt. (tok/s)	Avg. Latency (s)	TTFT avg (s)	ITL avg (ms)
1P-1D	3.78	966.6	25.6	20.2	21.4
1P-2D	4.13	1058.3	22.9	17.0	23.1
2P-1D	6.31	1616.4	14.9	7.8	27.9
2P-2D	7.96	2037.5	11.8	7.2	18.2

Note: GPU-level utilization is not reported here; the captured telemetry is host-wide (all 8 GPUs) rather than isolated per serving worker, so it cannot be attributed cleanly to a specific Prefill or Decode worker.

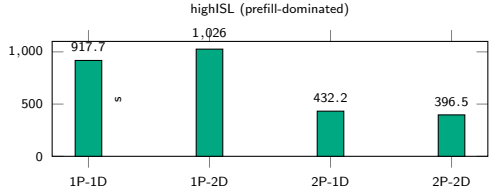
Performance Comparison Across Disaggregation Ratios (cont.)

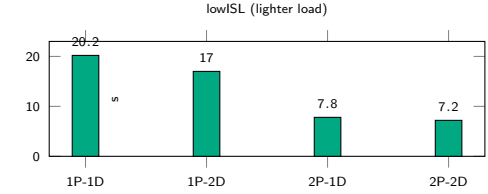
Output Token Throughput by Configuration





Average TTFT by Configuration





Introduction to AIPerf

Introduction to AIPerf

What is AIPerf?

- AIPerf is a benchmarking tool used to measure the performance of LLM inference servers.
- It generates synthetic client requests against an OpenAI-compatible endpoint and records timing statistics for each request.

Why is it Used?

- It provides a standardized, reproducible way to evaluate latency and throughput across different serving configurations.
- It helps identify bottlenecks in prefill, decode, and scheduling stages of an LLM serving deployment.

Key Metrics Captured

- **Time to First Token (TTFT)**
  Time from request submission until the first output token is received.
- **Time to Second Token (TTST)**
  Time from request submission until the second output token is received.
- **Inter-Token Latency (ITL)**
  Average time gap between consecutive output tokens during decode.
- **Output TPS**
  Number of output tokens generated per second by the server.
- **Request Throughput (req/s)**
  Number of complete requests the server processes per second.

Input Parameters

```
aiperf profile \
  --model meta-llama/Meta-Llama-3.1-8B-Instruct \
  --endpoint-type chat \
  --endpoint /v1/chat/completions \
  --url http://localhost:8000 \
  --streaming \
  --synthetic-input-tokens-mean 500 \
  --output-tokens-mean 256 \
  --request-count 500 \
  --num-dataset-entries 500 \
  --warmup-request-count 10 \
  --warmup-concurrency 10 \
  --slice-duration 10 \
  --gpu-telemetry pynvml \
  --server-metrics http://localhost:8000/metrics \
  --concurrency 100
```

GPU Telemetry & Server Metrics

GPU Telemetry Metrics (via pynvml)

- **GPU Utilization (%)**
  Percentage of time the GPU compute cores were actively processing work.
- **GPU Memory Used**
  Amount of GPU memory occupied by the model weights, KV cache, and activations.
- **Power Usage (W)**
  Instantaneous power draw of the GPU during the benchmark run.
- **GPU Temperature**
  Die temperature of the GPU, used to detect thermal throttling under load.

Server Metrics (via /metrics endpoint)

- **Running/Waiting Requests**
  Number of requests currently being processed versus queued on the server.
- **KV Cache Usage (%)**
  Fraction of the server's KV cache memory pool currently occupied.
- **Batch Size**
  Number of requests batched together by the scheduler for a given step.

Output Results (LLM Matrics)

NVIDIA AIPerf | LLM Metrics

Metric	avg	min	max	p99	p90	p50	std
Time to First Token (ms)	623.68	62.92	3,605.04	3,460.23	1,769.47	395.84	767.36
Time to Second Token (ms)	104.53	0.00	195.93	193.41	177.63	124.31	55.71
Time to First Output Token (ms)	623.68	62.92	3,605.04	3,460.23	1,769.47	395.84	767.36
Request Latency (ms)	10,725.85	838.97	13,681.11	13,582.21	13,381.02	11,176.48	2,557.18
Inter Token Latency (ms)	41.81	21.31	162.13	51.63	50.84	41.33	10.76
Output Token Throughput Per User (tokens/sec/user)	25.35	6.17	46.92	46.10	35.06	24.20	6.59
Output Sequence Length (tokens)	244.50	8.00	257.00	256.00	256.00	256.00	33.78
Input Sequence Length (tokens)	500.00	500.00	500.00	500.00	500.00	500.00	0.00
Output Token Throughput (tokens/sec)	2,225.75	N/A	N/A	N/A	N/A	N/A	N/A
Request Throughput (requests/sec)	9.10	N/A	N/A	N/A	N/A	N/A	N/A
Request Count (requests)	500.00	N/A	N/A	N/A	N/A	N/A	N/A

Figure: LLM Matrics

Output Results (GPU Telemetry)

localhost | GPU 2 | NVIDIA A100-SXM4-80GB

Metric	avg	min	max	p99	p90	p50	std
GPU Power Usage (W)	343.05	91.86	418.00	415.99	404.18	338.92	69.51
Energy Consumption (MJ)	0.02	N/A	N/A	N/A	N/A	N/A	N/A
GPU Utilization (%)	95.00	0.00	100.00	100.00	100.00	100.00	22.36
GPU Memory Used (GB)	82.67	82.56	82.80	82.79	82.76	82.66	0.08
GPU Temperature (_C)	51.00	42.00	56.00	55.81	55.00	51.00	3.60
Memory Utilization (%)	51.40	0.00	71.00	71.00	71.00	54.00	17.77
Decoder Utilization (%)	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Encoder Utilization (%)	0.00	0.00	0.00	0.00	0.00	0.00	0.00
JPEG Utilization (%)	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Power Violation (us)	0.00	N/A	N/A	N/A	N/A	N/A	N/A

Figure: GPU telemetry

KVBM and LMCache in Disaggregated Serving

The KV Cache Bottleneck

Memory Constraints in LLM Serving

- Large Language Model (LLM) serving architectures face severe memory bottlenecks due to the growing size of the Key-Value (KV) cache, particularly during long-context inference (e.g., 16,000 input tokens).
- Without intervention, the KV cache rapidly exhausts local GPU VRAM, leading to Out-Of-Memory (OOM) failures and request queue deadlocks.

Serving Architectures

- **Aggregated Serving:** Prefilling and decoding occur on the same GPU.
- **Disaggregated Serving:** Prefill and decode stages are isolated to dedicated GPUs (e.g., 2P2D topology), transferring KV cache across NIXL.

KV Buffer Manager (KVBM): Reactive Safety Net

What is KVBM?

- KVBM acts as a tiered memory manager, allowing the system to reactively evict excess KV tensors from GPU VRAM to external storage tiers when traffic is heavy.
- **Vertical Tiering:** Acts as a high-speed swap space in aggregated setups.
- **Horizontal Scaling:** Utilizes the NIXL connector to stream tensors across the network in disaggregated setups.

Evaluated Offloading Tiers

- **Baseline:** No offloading (relies strictly on local VRAM).
- **CPU Offloading:** Utilizes page-locked host memory via the PCIe bus.
- **Disk Offloading:** Utilizes persistent disk storage for massive capacity scaling.

LMCache: Proactive Prompt Reuse

What is LMCache?

- While KVBM handles overflow, LMCache is a proactive KV cache strategy. It offloads prefill KV blocks to a host-resident CPU DRAM pool.
- **Prefix Matching:** It reloads these blocks on prefix matches, allowing recurring prompts to bypass GPU prefill computation entirely.

Integration in Dynamo

- Enabled by composing `LMCacheConnectorV1` with `NixlConnector` under a `MultiConnector` on prefill workers.
- Deployed with large local CPU pools (e.g., 100 GB) and defined chunk sizes (e.g., 256 tokens).

Disaggregated Deployment Configurations

Configuring the Prefill Workers (2P2D Topology)

```
DYN_KVBM_CPU_CACHE_GB=100 \
LMCACHE_MAX_LOCAL_CPU_SIZE=100 \
CUDA_VISIBLE_DEVICES=6 python -m dynamo.vllm \
  --model meta-llama/Meta-Llama-3.1-8B-Instruct \
  --disaggregation-mode prefill \
  --kv-transfer-config '{kv_connector: MultiConnector ...}' \
  --discovery-backend file
```

Native Dynamo Variables

Offloading tiers and caching limits are toggled using native prefix variables like `DYN_KVBM_CPU_CACHE_GB` and `LMCACHE_MAX_LOCAL_CPU_SIZE`.

Performance Results: KVBM Impact

AIPerf Benchmarking (TTFT Reduction)

KVBM delivered dramatic reductions in Time To First Token (TTFT) by preventing queue deadlocks and allowing continuous processing:

Configuration	TTFT Reduction (%)
1P1D (ISL=500)	95.3%
2P2D (ISL=500)	90.6%
1P1D (ISL=16000)	85.5%
2P2D (ISL=16000)	83.0%

Note: While TTFT decreased massively, Inter-Token Latency (ITL) occasionally increased due to the overhead of transferring KV cache from the prefill to decode stages over NIXL.

Performance Results: LMCache & Prompt Reuse

The Importance of NDE (Number of Dataset Entries)

LMCache's effectiveness relies entirely on prompts recurring. Evaluated using a 2P2D layout with ISL=1000:

Metric	NDE=50 (High Reuse)	NDE=500 (No Reuse)
Token Hit Rate (%)	**87.9%**	1.0%
TTFT (ms)	**418.68**	756.26
NIXL Transfer (GB)	**13.60**	66.69

Result: When prompts repeated (NDE=50), the hit rate spiked, TTFT improved, and NIXL network transfer volume plummeted as prefills were skipped.

Key Takeaways and System Synergy

Solving Two Sides of the Same Problem

- **KVBM (Reactive Overflow):** Essential for preventing OOM crashes when unique queries overwhelm the system. It handles the raw capacity problem by tiering memory to CPU or Disk.
- **LMCache (Proactive Optimization):** Essential for minimizing compute overhead and network transfer for repetitive tasks (like system prompts or document templates).

Conclusion

In a disaggregated serving architecture, KVBM and LMCache work synergistically. Together, they transform a fragile setup into a highly resilient, scalable, and latency-optimized LLM serving environment capable of handling massive long-context workloads.

Theoretical KV Cache calculation

Theoretical KV Cache Calculation

Motivation

- During autoregressive inference, each processed token stores a **Key (K)** and **Value (V)** tensor.
- These tensors are reused during decoding, eliminating repeated attention computations.
- For models using **Grouped Query Attention (GQA)**, the KV Cache depends on the number of KV heads rather than the hidden size.

KV Cache Formula (GQA Models)

$$\boxed{\text{KV Cache} = 2 \times L \times S \times N_{KV} \times D_{head} \times B}$$

where

- L = Number of transformer layers
- S = Total sequence length (Input + Output tokens)
- N_{KV} = Number of KV heads
- D_{head} = Head dimension
- B = Bytes per element (BF16 = 2 Bytes)
- Factor 2 accounts for both Key and Value tensors

Transformer Concepts for KV Cache

Key Concepts

- **Transformer Layer**
  - One processing block of the transformer.
  - Meta-Llama-3.1-8B contains **32 layers**.
- **Hidden Size**
  - Internal representation of each token.
  - Llama-3.1-8B uses a hidden size of **4096**.
- **Attention Heads**
  - Hidden vector is divided into multiple parallel heads.
  - 4096 / 32 = **128 values per head**.
- **Query (Q), Key (K), Value (V)**
  - Query searches for relevant information.
  - Key identifies stored information.
  - Value contains the information returned by attention.

KV Cache stores only the Key and Value vectors generated by the KV heads for every token in every transformer layer.

Example Calculation and Experimental Validation

Model Parameters

Parameter	Value
Model	Meta-Llama-3.1-8B
Layers (L)	32
KV Heads (N_{KV})	8
Head Dimension (D_{head})	128
Input Tokens	16000
Output Tokens	256
Sequence Length (S)	16256
Precision	BF16 (2 Bytes)

$$2 \times 32 \times 16256 \times 8 \times 128 \times 2$$

$$= 2.13 \text{ GB}$$

Experimental Results

Metric	Value
Theoretical KV Cache	2.13 GB
Measured NIXL Transfer	2.10–2.15 GB
Maximum Transfer	2.147 GB
Agreement	Close Match

Type	Metric	Unit	avg	min	max	std	p1	p5	p10	p25	p50
histogram	vllm:nixl_bytes_transfe		500	1.2835	1.05E+12	2.7E+09	2.1E+09	1.49E+09	1.56E+09	1.64E+09	1.88E+09
histogram	vllm:nixl_bytes_transfe		0								
histogram	vllm:nixl_bytes_transfe		0								

AI Perf NIXL transfer measurements for ISL = 16000.

Key Observation

The measured NIXL transfer closely matches the theoretical KV Cache size, validating the analytical model for Meta-Llama-3.1-8B using Grouped Query Attention.

AI Configurator

Overview

What is AI Configurator?

- AI Configurator is a tool that recommends inference deployment configurations for Large Language Models (LLMs).
- Recommendations are based on model, hardware, and workload parameters.

Supported Modes

- **CLI (Command Line Interface):** A terminal-based interface that outputs recommended configurations and stores results in a specified directory.
- **Web Application:** An interactive Gradio-based interface accessible through a browser. It runs on port 7860 and requires port forwarding on HPC systems.

Database Modes

Available Lookup Modes

- **EMPIRICAL Mode:** Uses pre-collected benchmark data. Provides the fastest and most accurate recommendations when benchmark data exists.
- **HYBRID Mode:** Combines empirical benchmark data with analytical models. Used when complete benchmark data is unavailable.
- **ANALYTICAL Mode:** Uses analytical models only. Suitable when no benchmark data exists.

Best Practice

EMPIRICAL mode is preferred when benchmark data is available.

CLI Mode Example

Example Command Structure (EMPIRICAL Mode)

```
export HF_TOKEN="<token>"
aiconfigurator cli default \
  --model-path meta-llama/Meta-Llama-3.1-8B-Instruct \
  --total-gpus 8 \
  --system a100_sxm \
  --backend vllm \
  --backend-version 0.12.0 \
  --database-mode EMPIRICAL \
  --isl 500 \
  --osl 256 \
  --save-dir ./aic-results-llama
```

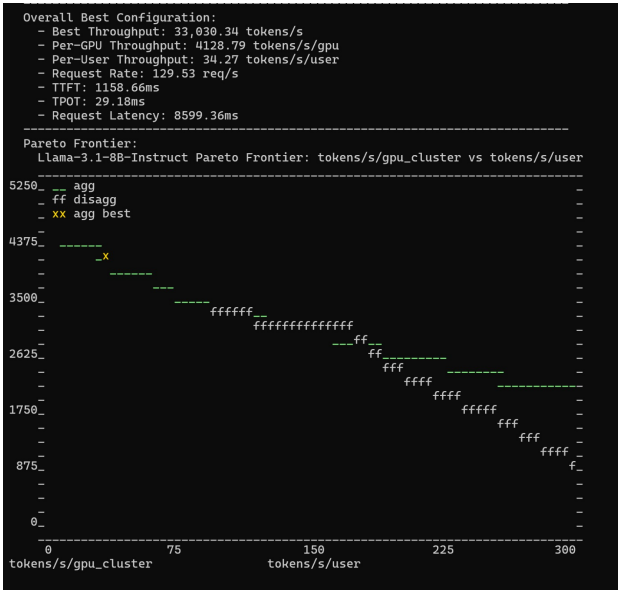
Key Arguments Explained

- `--isl` and `--osl` specify the Input Sequence Length and Output Sequence Length in tokens.
- `--save-dir` dictates the directory where results are stored.

AI CONFIGURATOR Example

```
Overall Best Configuration:
  - Best Throughput: 33,030.34 tokens/s
  - Per-GPU Throughput: 4128.79 tokens/s/gpu
  - Per-User Throughput: 34.27 tokens/s/user
  - Request Rate: 129.53 req/s
  - TTFT: 1158.66ms
  - TPOT: 29.18ms
  - Request Latency: 8599.36ms
```

Pareto Frontier:
Llama-3.1-8B-Instruct Pareto Frontier: tokens/s/gpu_cluster vs tokens/s/user



Tested Configuration and Notes

Tested Configuration

- **Model:** meta-llama/Meta-Llama-3.1-8B-Instruct
- **Hardware:** NVIDIA A100-SXM4-80GB $\times$ 8 (System: a100_sxm)
- **Backend:** vllm v0.12.0
- **Workload:** 500 Input Tokens / 256 Output Tokens

Key Notes

- The HF_TOKEN must have read access to the requested Hugging Face model repository.
- The SSH tunnel must remain active while using the web application.
- AI Configurator estimates deployment configurations and does not execute model inference workloads.

Conclusion

- Successfully deployed **NVIDIA Dynamo** for Large Language Model (LLM) inference on an HPC system.
- Implemented and evaluated both **Aggregated** and **Disaggregated Serving** architectures.
- Validated the complete inference workflow, including request routing, worker execution, KV cache management, and response generation.
- Benchmarked system performance using **NVIDIA AI Perf**, analyzing latency, throughput, and KV cache transfer.
- Demonstrated that **Disaggregated Serving** improves scalability by separating **Prefill** and **Decode** operations.
- Showed that **KVBM** reduces Time To First Token (TTFT), while **LMCache** accelerates repeated prompts through prompt reuse.
- Explored the **AI Configurator** for recommending optimal deployment configurations based on model, hardware, and workload.
- Established a strong foundation for scalable LLM serving, with future scope for **multi-node deployments** using **HPE Slingshot Networking**.

Thank You!

Questions?